TestVDB: Detecting API Compliance Defects in Vector Database Systems via Contract-Truth Separation

Anonymous Author(s)
Submission #XXX

Abstract

Vector Database Management Systems (VDBMSs) underpin Large Language Model (LLM) applications, yet incorrect-behavior defects—43% of VDBMS bugs—lack practical oracles, as current fuzzers detect only crashes. We target *API compliance defects*: bugs where a VDBMS silently accepts inputs or behaviors that violate its documented contract. We present **TestVDB**, the first LLM-driven detector for them. TestVDB auto-derives a contract oracle from API documentation and applies **Contract-Truth Separation** (CTS), isolating LLM-generated assertions from a truth layer that falsifies them via maintainer-authority evidence; CTS is motivated by *contract hallucination propagation*—when one LLM family both generates a contract and judges compliance, hallucinated constraints are self-confirmed. TestVDB produced 111 issues across five VDBMSs; maintainers acknowledged 36 (28 fixed). On a controlled retrospective, the dev-reviewer's source anchor lifts false-positive suppression from 31% to 81% while retaining 96.7% of true positives.

CCS Concepts

• **Software and its engineering**; • **Information systems**;

Keywords

Vector database, API compliance defects, LLM-driven testing, test oracle, contract hallucination, multi-agent verification

ACM Reference Format:
Anonymous Author(s). 2026. TestVDB: Detecting API Compliance Defects in Vector Database Systems via Contract-Truth Separation. In *Proceedings of Proceedings of the ACM Conference (Conference'26).* ACM, New York, NY, USA, 8 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

1 Introduction

VDBMSs such as Milvus, Qdrant, Weaviate, Chroma, and MeiliSearch now underpin retrieval-augmented generation, recommendation, and multimodal search. As they enter production at scale, their reliability directly affects downstream decisions. A study of 1,671 bug-fix pull requests across 15 VDBMSs [25] finds that more than half of all defects are functional failures; a complementary testing roadmap [22] reports that *incorrect-behavior* bugs (43.0%) substantially outnumber crash/hang bugs (23.1%).

Conference'26, Location

ACM ISBN 978-1-4503-XXXX-X/2026/06
https://doi.org/10.1145/nnnnnnn.nnnnnnn

Table 1: Oracle candidates for API compliance defects.

Candidate oracle	Why it fails for compliance defects
Crash (VDBFuzz [23])	Compliance defects do not crash
PoC reproduction	No observable failure signal
Differential testing	No reference implementation for VDBMS APIs
Equivalence transformation	No query form to transform equivalently
LLM	**Only flexible semantic candidate—but it both generates the contract and judges, both unreliable**

Despite this distribution, current VDBMS testing skews toward the minority. VDBFuzz [23], the first dedicated VDBMS fuzzer, uses crash as its oracle and detects only crash/hang bugs, leaving the majority without a practical oracle. The roadmap [22] flags this as the central open challenge: incorrect behavior "poses significant challenges for test oracles," and approximate vector search renders traditional differential testing unsuitable. Its Future Work asks both for "an oracle for evaluating the correctness of vector search results" and for methods that let LLMs "stay updated with the latest syntax and functionalities."

We target a tractable slice of this gap: *API compliance defects*—bugs where the VDBMS silently accepts an input or produces a behavior that violates its documented contract (e.g., accepting `nprobe=0`, silently normalizing `shardsNum=-1`, or returning success where the docs prescribe rejection). These bugs do not crash, but they corrupt query semantics, lower recall, and expand the attack surface. Crucially, they admit an oracle where general result-correctness does not: the API contract itself.

Why an LLM, and why that is not enough. Compliance defects produce no observable failure signal, and VDBMS APIs have no widely-agreed reference implementation for differential testing. By exclusion (Table 1), the only candidate oracle capable of flexible semantic judgment over an undocumented boundary is an LLM. But using an LLM as the oracle introduces two unreliable roles: it *generates* the contract (from docs) and it *judges* compliance—and both can hallucinate.

Core reversal: the oracle itself may be wrong. The naive design—an LLM extracts a contract from docs, then the same family of models judges compliance against it—is brittle because of *contract hallucination propagation*. We observed a concrete instance: the formalizer "extracted" a complexity requirement from `constant.go` that is in fact a guard against a historical performance regression, not a behavioral constraint; the judge then confirmed violations against it. Across our submissions, 12 of the 48 non-rejected adjudicated issues (acknowledged or by-design, i.e., 52 adjudicated

minus 4 rejected; 25%) were marked by-design: the LLM-derived contract was stricter than the maintainers' true intent (demanding rejection of idempotent duplicate creates, eventual-consistency staleness, or lenient defaults). When the same model family generates the contract and judges compliance, hallucinated constraints are self-confirmed.

Approach: Contract-Truth Separation. When no mechanical oracle is available, the only usable truth source is *maintainer authority*—source code, prior PRs, by-design intent, issue history. It is scarce, so it must be proxied by an agent. We separate the *assertion layer* (LLM-generated contracts and judgments) from a *truth layer* and use the latter to falsify the former. The truth layer is a dev-reviewer agent that applies counter-evidence along three anchors: clean reproduction, source-grounded verification (the direct counter to hallucination), and threat-model cross-check. This mirrors how a maintainer adjudicates an incoming bug report.

Results. TestVDB produced 111 candidate issues across five VDBMSs (adjudicated signal concentrated on Milvus and Qdrant); maintainers acknowledged 36 (28 fixed, 8 accepted-open). On a controlled retrospective over all 52 maintainer-adjudicated candidates (same population), the dev-reviewer's source anchor lifts false-positive suppression from 31% to 81% while retaining 96.7% of true positives (n=30); aggregated over the five systems, maintainer-adjudicated precision is 69.2% (36/52), within [43.9%, 80.5%] under pending-resolution sensitivity (Section 5.3). We report the boundary honestly: 75% of yield is boundary/validation compliance; crash bugs are excluded by design (complementary to VDBFuzz); soft result-correctness (ANN recall, ranking) remains open.

Positioning. We respond to VDBFuzz's generation-side Future Work (LLM-generated diverse API interactions and staying current with evolving APIs). Our judgment side provides a contract oracle that extends detection beyond crash into the API-compliance subset of incorrect behavior. We do *not* claim to solve VDBFuzz's oracle-for-correctness direction (result correctness of vector search), which remains open [22].

Contributions.

(1) The **first LLM-driven realization and empirical study of VDBMS API-compliance defect detection**: TestVDB is the first end-to-end system for non-crash compliance defects on the agenda of [22], producing 111 submissions across five VDBMSs (36 maintainer-acknowledged, 28 fixed) with a reproducible open-source dataset. Adjudicated precision is validated on Milvus and Qdrant; three further VDBMSs serve as breadth probes on the attack surface.
(2) **Contract-Truth Separation (CTS)**: a design principle that isolates LLM-generated assertions from a maintainer-authority truth layer and falsifies them via source-grounded verification, motivated by *contract hallucination propagation*—when one LLM family both generates a contract and judges compliance, hallucinated constraints are self-confirmed (25% of adjudicated submissions were by-design). The dev-reviewer's source anchor lifts FP suppression from 31% to 81% at 96.7% TP retention; the threat-model anchor is ablated as a noisy complement (§5.4).

Table 2: TestVDB scope projected onto the roadmap symptom taxonomy [22].

Symptom class (share)	TestVDB coverage
Crash/Hang (23.1%)	Out of scope (L1 gate filters; 1 incidental in yield)
Incorrect Behavior (43.0%)	**In scope**: boundary/validation (75% of yield) + diagnostic + state/logic subsets
Performance (9.3%)	Out of scope
Build (9.7%)	Out of scope

(3) A **model-free invariant oracle subclass**—`COSINE` distance > 1.0 for identical vectors, incomplete index results, payload-filter violations—that violates hard mathematical bounds, needs no LLM judgment, and reproduces across Milvus and Qdrant; it is adoptable independent of TestVDB's LLM pipeline.

2 Background and Problem Formulation

2.1 VDBMS Architecture

A VDBMS stores, indexes, and queries dense vector embeddings across a storage engine, a vector index (HNSW, IVF, etc.), a query processor (search, filtered/predicated search, multi-vector search), and client SDKs exposing REST and gRPC APIs [22]. Our targets live at the API surface and the query-processing logic behind it.

2.2 Bug Taxonomy and Our Scope

We adopt the symptom-level taxonomy of [22] as our primary frame (Crash/Hang 23.1%, Incorrect Behavior 43.0%, Performance 9.3%, Build 9.7%) and corroborate it with the larger empirical study [25] (5 symptom categories, 31 fault patterns) used as a fault-pattern vocabulary. Our compliance-dimension axis projects onto the symptom axis as in Table 2: we operate inside Incorrect Behavior, specialize in its boundary/validation subset, and exclude Crash (by design), Performance, and Build.

2.3 API Compliance Defects

An *API compliance defect* is a behavior where, for an input governed by a documented or implementable contract, the VDBMS (i) accepts an input the contract prescribes to reject (*illegal success*), (ii) rejects or mishandles an input the contract prescribes to accept, or (iii) returns a response whose status, payload, or side effects violate the contract (*poor diagnostics / state violation*). The contract is not assumed perfect; Section 4 confronts contract unreliability directly.

2.4 The Oracle Problem

Compliance defects produce no crash and no exception, and—unlike SQL—there are no widely-agreed reference semantics for VDBMS APIs across vendors. Differential testing is unsuitable because APIs diverge and results are approximate [22]. Metamorphic relations [21] serve result correctness but not API-acceptance decisions. The contract itself is the only candidate oracle—but it must be obtained and trusted, which is the core problem we address.

3 Approach

3.1 Overview

TestVDB is a five-stage pipeline (Figure 1): (1) contract extraction from API documentation via an LLM knowledge-extractor; (2) attack generation by specialized agents guided by a threat-model prior; (3) a four-judge debate producing Stage-2 candidates; (4) a dev-reviewer that applies Contract-Truth Separation as counter-evidence; (5) a two-layer novelty gate. A threat-model artifact is reused across stages as an experience prior.

Implementation. All agents are LLM-driven and served by GLM-5.2; four heavy-reasoning agents (orchestrator, dev-reviewer, threat-modeler, bug-shape-extractor) run on the high-budget configuration and the remaining sixteen on the low-budget configuration—both the same GLM-5.2 backbone, differing in context window and max output tokens rather than model families. Four independent judges (evidence, novelty, severity, documentation) score each Stage-2 candidate; the orchestrator aggregates their verdicts and forwards survivors to the dev-reviewer, which suppresses a candidate if any of its three anchors (clean reproduction, source-grounding, threat-model) yields counter-evidence. An end-to-end trace (contract hallucination → source-grounded reclassification) appears in §4.

Reproducibility and cost. Target VDBMS versions are pinned per system (Milvus 2.6.19 for the single-LLM and single-layer ablations; the full version matrix is in the artifact). Agents inherit the Claude Code runtime's default sampling configuration with no explicit temperature override, and each of the 20 agents is defined by a task-structured role prompt (full prompts in the artifact). A full pipeline run per target invokes the 20 agents over $O(10–50)$ generated attack candidates; wall-clock is dominated by the dev-reviewer's source-grounding step (repository clone and source retrieval) and live Docker re-probes rather than raw LLM latency. The 111-submission study plus three ablation batches (52-candidate retrospective, 15-probe single-layer A1, 15-probe single-LLM B9) constitutes the full LLM-call budget; in aggregate this is on the order of 10^4 LLM calls ($\sim 10^7$ tokens); per target, a full pipeline (contract extraction through dev-reviewer) costs on the order of 10^3 calls ($\sim 2\times10^6$ tokens) and completes in a few hours (on the order of $10 per target at current LLM API pricing, comparable to a few hours of manual boundary testing). For this cost, TestVDB's marginal value over a hand-written boundary fuzzer is threefold: non-boundary yield (state/logic, diagnostic, result-correctness probes), source-grounded FP-suppression across all categories, and spec-gap detection where the doc is silent (§5.3). Precise per-token and wall-clock accounting is part of the anonymized artifact at https://anonymous.4open.science/r/testvdb-anon-D644/, to be made public on acceptance.

3.2 Contract Extraction and the Threat-Model Prior

A knowledge-extractor agent crawls official API documentation and emits structured contracts (parameter domains, enum values, documented limits, status-code semantics) that seed both attack agents and judges. A threat-model artifact encodes recurring by-design intents (idempotency, eventual consistency, lenient defaults) and known blind spots. It is *designed* to be injected at generation, judgment, and dedup; however, in our runs the blindspot indicators were never populated and its effect could not be isolated (Section 5.4), so we treat it as an unvalidated optional prior rather than a working component of the evaluated system.

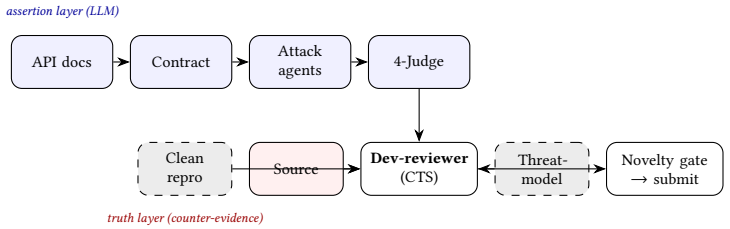

Figure 1: TestVDB pipeline. The assertion layer (LLM contract + 4-judge) is falsified by the dev-reviewer's three counter-evidence anchors (truth layer): the *source* anchor (solid) is the primary, validated anchor; the *threat-model* anchor (dashed) is ablated in §5.4 as a noisy offline complement (catches boundary-default residuals source misses, but over-fires on state/concurrency); the *reproduction* anchor (gray, dashed) remains design-level and not yet evaluated. CTS = Contract-Truth Separation.

3.3 Attack Generation and the Four-Judge Debate

Three attack agents target the compliance dimensions: a *boundary* agent (illegal-success at parameter, enum, and limit edges—the dominant yield), a *semantic* agent (diagnostic-quality violations), and a *state* agent (state/logic inconsistencies). Candidates enter a four-judge debate (evidence/severity/novelty/doc axes) drawing on multi-agent verification ideas [7]. The Stage-2 output of this debate feeds the dev-reviewer.

3.4 Contract-Truth Separation and the Dev-Reviewer

Single-layer LLM judgment is unreliable because the same model family generates the contract and judges compliance. CTS breaks this self-confirmation by introducing an independent truth layer. The dev-reviewer agent proxies maintainer authority and falsifies each Stage-2 candidate along three anchors:

- **Clean reproduction.** Build a minimal reproducer against a live VDBMS of the pinned target version; reject candidates whose "defect" is a tooling artifact (we withdrew early candidates that misread a missing `rowCount` field as "returns −1").
- **Source-grounded verification.** Locate the implementation behind the API and check whether the observed behavior is intended—the direct counter to contract hallucination (Section 4).
- **Threat-model cross-check.** Match against known by-design intents before flagging a violation.

This mirrors maintainer adjudication: most of our false positives stem from contracts stricter than true intent, and source grounding is what exposes them. Of the three anchors, we validate source-grounding as the primary anchor in the controlled retrospective

(§5.3); the threat-model anchor is ablated as a noisy complement (§5.4); the reproduction anchor remains design-level future work.

3.5 Novelty Gate

A two-layer gate prevents re-reporting: L1 checks local history and the threat-model blind-spot set; L2 queries the target repository's issues and PRs. Candidates surviving both are submitted.

4 Contract Hallucination Propagation

We highlight a phenomenon that, to our knowledge, has not been characterized in LLM-driven testing. When an LLM generates the contract *and* judges compliance, hallucinated constraints are not caught by the judge—they are produced by the same generation–judgment family and thus inherit mutual confirmation. We observed two forms.

(1) Fabricated provenance. The formalizer produced a "complexity requirement" attributed to `constant.go`; source grounding showed the constant guards a past performance regression, not a behavioral constraint. The Stage-2 judge nonetheless confirmed violations against it.

(2) Over-strict intent. 12 of the 48 substantively adjudicated submissions (25%) were marked by-design: the contract demanded rejection of behaviors maintainers intended to allow—idempotent duplicate creates returning success, eventual-consistency staleness, lenient naming defaults, and silent enum/empty substitutions.

Formally, let C_{LLM} be the LLM-derived contract and C_{true} the maintainer intent. Single-layer judgment assumes $C_{\text{LLM}} = C_{\text{true}}$; when $C_{\text{LLM}} \supset C_{\text{true}}$ (stricter), genuine by-design behaviors are flagged as violations, and the judge—sharing the generator's bias—does not dissent. The dev-reviewer's source-grounded anchor is the mitigation: it reintroduces an external truth signal (C_{true} proxied by source and history). We present this as a qualitative characterization; a quantitative study of hallucination frequency is future work, and the 12 by-design cases are direct observations rather than a measured rate over all inputs.

5 Evaluation

We address three primary research questions (RQ1–RQ3) and report one exploratory negative result (RQ4, Section 5.4).

5.1 RQ1: Detection Capability

We ran TestVDB against five VDBMSs, pinning exact target versions (Table 3). Yield concentrates on Milvus (51 submissions) and Qdrant (26); MeiliSearch (3) and Chroma (1) contribute near-zero adjudicated signal, so our cross-system generalization is claimed primarily for Milvus and Qdrant, with Weaviate, MeiliSearch, and Chroma as breadth rather than statistical evidence.

Defect-type distribution. Of the 36 acknowledged true positives, 27 (75%) are boundary/validation, 3 (8%) diagnostic-quality, 2 (6%) state/logic, 1 (3%) crash, and 3 (8%) result-correctness. The result-correctness cases are instructive: they include `COSINE` distance returning > 1.0 for identical vectors (a hard mathematical-invariant violation, reproduced on both Milvus and Qdrant), incomplete index results (2 of 25 matching points returned), and a payload filter returning points with a missing field. These are detectable because they violate expressible invariants; soft result-correctness (ANN recall/ranking error) remains open.

Table 3: Issues submitted and maintainer outcomes (5 VDBMSs). 52 of the 111 submissions have been adjudicated (36 acknowledged = 28 fixed + 8 accepted, 12 by-design, 4 rejected). Of the remaining 59, 30 are *pending* (open, awaiting triage) and drive the sensitivity interval in Section 5.3; 29 are *excluded* (closed-no-label or duplicate, unresolvable) and lie outside that interval.

VDBMS	Sub.	Fix.	Acc.	ByD.	Rej.	Pend.	Excl.
Milvus	51	14	8	12	0	0	17
Qdrant	26	11	0	0	3	8	4
Weaviate	30	3	0	0	1	21	5
MeiliSearch	3	0	0	0	0	0	3
Chroma	1	0	0	0	0	1	0
Total	**111**	**28**	**8**	**12**	**4**	**30**	**29**

Complementarity with VDBFuzz. Our yield is almost entirely non-crash (35/36); VDBFuzz's crash oracle—its title frames the problem as "Crash Bugs" [23] and its source contains no compliance-checking logic—cannot detect non-crash defects, so at most 1/36 of our yield (the single crash case) could overlap with VDBFuzz's targets. Conversely, our L1 gate deliberately filters crash-prone inputs, so TestVDB does not target the crash bugs VDBFuzz finds. The complementarity follows from the oracle definitions; a head-to-head empirical comparison on identical targets is future work.

5.2 RQ2: Case Studies

We trace three end-to-end cases that span the TP/FP boundary and the dev-reviewer's role, then isolate a model-free oracle subclass. **True positive, correctly surfaced (Milvus #47729).** An invalid `nprobe=0` was accepted by search. The attack agent derived the probe from the contract; the four-judge debate confirmed it; the dev-reviewer's source anchor raised no counter-evidence (no documented lower bound); the maintainer fixed it. **True positive, correctly surfaced (Milvus #49844).** An empty filter silently triggered a full scan; same path; maintainer accepted. **False positive, correctly suppressed (Milvus #50193).** `get_stats` returned `rowCount=0` immediately after insert. The four-judge confirmed it as a bug; the dev-reviewer's source anchor reclassified it as documented eventual consistency (async aggregation; 0 is a valid intermediate)—precisely the consistency-semantics FP class the source anchor targets—and the maintainer later closed it as by-design, confirming the reclassification.

Model-free invariant oracles. The `COSINE` distance returned > 1.0 for identical vectors, independently reproduced on both Milvus and Qdrant. Unlike the contract-driven cases above, this violates a *mathematical invariant* (cosine similarity is bounded in $[-1, 1]$) and needs no LLM judgment—an expressible, model-free oracle subclass worth distinguishing from contract oracles. The same class catches incomplete index results (2/25 matching points returned) and payload filters returning points with a missing field. This subclass is the paper's most defensible technical finding: it depends on no LLM

judgment, reproduces across vendors, and violates a hard mathematical bound, so it is the least contingent on the agent-design choices the rest of the evaluation rests on.

5.3 RQ3: Precision — Controlled Retrospective and Aggregate

5.3.1 Controlled Retrospective. We re-triaged all 52 maintainer-adjudicated candidates (36 TP, 16 FP) under two blind conditions on the *same* population: claim-only (4-judge layer) vs. source-grounded (dev-reviewer's source anchor), outcomes hidden via label-isolated agents. Source-grounding lifts FP suppression from 5/16 (31%) to 13/16 (81%, 2.6×)—Milvus 33%→75%, non-Milvus 0%→100%—while retaining 29/30 TPs (96.7%; 6 TPs were unreachable via API rate limits). This same-population comparison is our primary evidence for the dev-reviewer's contribution.

5.3.2 Aggregate Precision. Across five VDBMSs, 52/111 submissions are adjudicated (36 acknowledged, 12 by-design, 4 rejected). Counting by-design and rejected as maintainer-judged FPs,

$$\text{precision}_{\text{adj}} = \frac{\text{acknowledged}}{\text{acknowledged + by-design + rejected}} = \frac{36}{36 + 12 + 4} = 69.2\%$$

This is an end-to-end operating point, not the "after" arm of the retrospective above (different population and denominator).

5.3.3 Sensitivity Analysis. Of 59 unadjudicated submissions, 30 are pending and 29 are excluded (closed-no-label or duplicate). As a bound on the 29 excluded: if all were false positives, precision over the 81 non-pending drops to 36/81=44.4%; if all were true positives, it rises to 65/81=80.2%. For the 30 pending: all-valid gives (36+30)/82 = 80.5%, all-invalid gives 36/82 = 43.9%. Aggregate precision therefore lies in [43.9%, 80.5%] (69.2% point estimate).

5.3.4 Baseline Comparisons. **Within-system (same 52 pool).** Claim-only lets 11/16 FPs through (judgment-layer precision 33/44 = 75%); source-grounded lets 3/16 through (29/32 = 91%).[1]

Single-layer counterfactual. Removing the FP-suppression chain and re-running generation + 4-judge debate on milvus v2.6.19 confirmed 3 candidates (2 TP: #47729 invalid `nprobe=0`, #49844 empty-filter full scan; 1 FP: #50193 eventual-consistency). To bound the arm's FP inflow without LLM-proxy judgment, we re-probed all 27 dev-reviewer-killed candidates live on a fresh v2.6.19 container and source-grounded each via milvus constants (`DefaultConsistencyLevel=ClBounded`, `DefaultShardNumber=0`); all 27 reproduce as true FPs (*27/27 live, over-kill* 0/27). Combining the 27 suppressed with the 36/52 baseline gives single-layer precision 36/(36+16+27) = 45.6% vs. TestVDB's 69.2%. This 45.6% rests on live + source grounding (not LLM-proxy); we report it as a directional lift at zero recall cost, treating the same-population 31% → 81% as the cleaner head-to-head.

Single-LLM. A single LLM end-to-end (no multi-agent debate, no CTS) produced 51 candidates, 13 LLM-judged TP (25.5%, Wilson [15.5%, 38.9%]); a source-grounded self-judgment filter drops it to 11.8% (6/51, [5.5%, 23.4%]) after by-design reclassification. Both remain far below 69.2%, so multi-agent debate plus CTS contributes beyond source anchoring alone.

[1]The two conditions use slightly different TP denominators (claim-only scored all 36; source-grounding scored 30 reachable via the GitHub API, 6 rate-limited), within the same 52-candidate pool.

Table 4: Precision/ground-truth comparison across arms, grouped by truth tier (weak proxy → maintainer gold). Rows are *not* directly comparable across tiers (different ground truths); the retrospective and maintainer tiers share the same 52-candidate pool. CI is Wilson 95% *except* the last row, whose [43.9%, 80.5%] is the pending-resolution sensitivity interval (Section 5.3), not a Wilson CI. †The schema-fuzzer 37% is a probe→accept rate (7 of 19 probes surfaced an API-accepted candidate), not a candidate precision; source-grounded post-filter classifies 5 of those 7 as genuine violations (71%, live-confirmed on a fresh v2.6.19 container).

Arm	Precision	*n*	Ground truth (tier)	95% CI
LLM-judged (weak proxy)				
Single-LLM	25.5%	51	LLM self-judgment	[15.5%, 38.9%]
Single-LLM + source filter	11.8%	51	LLM + source	[5.5%, 23.4%]
API-acceptance (weak proxy)				
Schema fuzzer (no LLM)	37%†	19	API-acceptance	—
Retrospective (same 52-candidate pool, blind judge)				
Single-layer 4-judge	75%	44	claim-only re-triage	—
TestVDB source-grounded	91%	32	source re-triage	—
Maintainer adjudication (gold)				
TestVDB (end-to-end)	69.2%	52	maintainer triage	[43.9%, 80.5%]

Schema-aware boundary fuzzer. A hand-written boundary fuzzer (19 probes, no LLM) surfaced 7 API-accepted candidates; source-grounded post-filter classifies 5 as genuine violations (71%) and 2 as by-design defaults. This concedes that on the boundary/validation subset (75% of yield) a spec-driven fuzzer is effective. TestVDB's marginal value lies where the fuzzer cannot reach: (a) non-boundary probes (state/logic, diagnostic, result-correctness: 8/36 TPs); (b) CTS FP-suppression (the fuzzer has no source-grounded layer); (c) spec-gap bugs where the doc is silent. A Schemathesis head-to-head is blocked (Milvus serves no standards-compliant OpenAPI; `/swagger`, `/openapi.json` all 404).

Rows across arms use different ground truths (LLM self-judgment / API-acceptance / blind re-triage / maintainer), so they are *not* directly comparable across tiers; Table 4 and Figure 2 make the asymmetry explicit rather than drawing cross-tier conclusions.

5.3.5 Anchor Attribution. On the 16 FPs, the *source* anchor drives the full lift (claim-only 5/16 = 31%; adding source 13/16 = 81%); the 3 residuals are silent-absent cases (no validation code to cite). The *threat-model* anchor (§5.4): source-alone 9/12, threat-alone 6/12 (unstable), union 11/12, TP 4/4 throughout—threat catches 2 source residuals (boundary-default) but over-fires on state/concurrency (BS-03/06). The *reproduction* anchor needs a live container and is not exercised here. Source is primary; threat-model is a noisy complement; reproduction is future work.

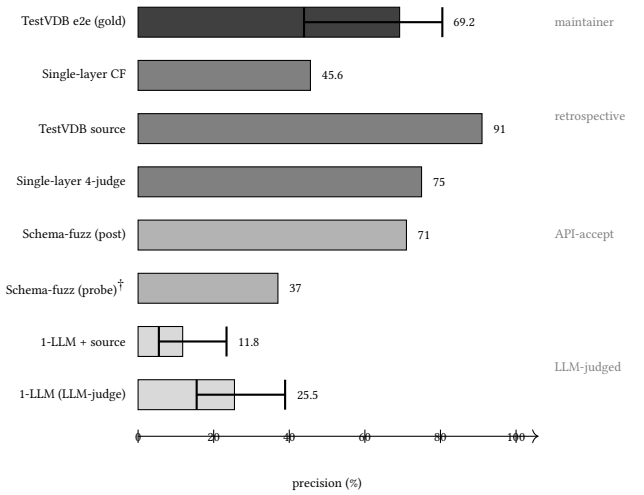

Figure 2: Precision by ground-truth tier (darker = stronger). Whiskers = Wilson 95% CI; † = probe→accept rate, not candidate precision. Details in Table 4.

5.4 RQ4: Threat-Model Anchor — Wired and Ablated

An earlier double-blind ablation on Milvus v2.6.17 (control with threat-model cognition vs. experiment without; n=5 TP, 7 FP) returned an exploratory negative (TP recall 20%/60%; blindspot indicators never reached the dev-reviewer's consumption path; the agent was unstable across runs). On inspection, the negative was confounded by a wiring gap: the threat-modeler populated `threat_model.json` with 6 cognitive blindspots and 10 by-design behaviors, but the dev-reviewer consumed `developer_cognition.json`'s `blindspot_indicators` field, which was empty. We re-ran the ablation with the wiring fixed (a v2 dev-reviewer prompt that reads `threat_model.json` directly), on the 12 Milvus FPs of the retrospective plus 4 Milvus TPs as a recall control, under three conditions: *source-alone* (network on, threat-model blanked), *threat-alone* (network off, threat-model populated), and *both* (union of the two independent verdicts). Source-alone suppresses 9/12 FPs (75%); threat-alone suppresses 6/12 (50%) and is unstable across runs (one boundary FP flipped between runs); their union (a candidate is suppressed if *either* anchor independently flags it—an OR of two blind verdicts, not a joint dispatch requiring both to agree) suppresses 11/12 (92%); all three conditions retain 4/4 TPs. The threat anchor catches 2 of source's 3 residuals (boundary-default cases where no validation code exists to cite) but misses 5 state/concurrency FPs that source catches—its blindspot mechanism (BS-03 concurrency, BS-06 state-consistency) over-fires there, pushing genuine by-design FPs toward CONFIRMED. The threat anchor is thus neither an offline substitute for source nor dead weight, but a *noisy complement* whose value is the union. We do not claim the three-anchor design as a clean validated contribution on the strength of n=12; we report it as a diagnosed result that explains why source-grounding remains the primary anchor and bounds the threat anchor's standalone effect honestly.

5.5 Threats to Validity

- **Internal.** Maintainer acknowledgment is a weak ground truth; triage may reflect report clarity rather than defect validity.
- **Selection.** The 111 submissions were filtered from a larger candidate pool by the novelty gate; the candidate-to-submission ratio is not instrumented, leaving submission-selection bias roughly bounded.
- **External.** The same-population ablation (RQ3) is Milvus-plus-Qdrant only; Weaviate submissions are largely undiagnosed.
- **Construct.** Defect-type classification is title-based, pending per-defect confirmation.
- **LLM variance.** Re-adjudicating the 46 source-grounded candidates five times with independent agents yields 99.1% pairwise agreement and 45/46 unanimous verdicts; residual variance sits upstream in version-pinned source extraction.
- **Contamination.** GLM-5.2 may have seen pre-2024 Milvus source in training. A memorization canary (bare model, no docs) recalls general bug-class knowledge (e.g., "cosine $\in [-1, 1]$") but *0/9* held-out issues at issue-specificity, so the 9-bug cohort is not memorization-confounded; we flag the cosine>1 invariant as the one overlap with general mathematical knowledge rather than count it as independent evidence. A contract counterfactual (DeepSeek on the same doc passages) reproduces over-strict constraints in 2/9 of an expanded N=10 set—partly task-intrinsic on default-as-value cases but predominantly GLM-specific, which CTS targets directly.
- **Recall scope.** The 96.7% figure is judgment-layer TP retention, not end-to-end discovery recall. Two held-out-bug pilots bound where TestVDB applies: a *spec-completeness* limit (qdrant 1.5.0 silently drops dimension-mismatched points, which the OpenAPI spec does not describe) and a *version-pinning* limit (milvus cosine>1.0 was fixed before the release version, so bug-present versions must be pinned to the exact dev/patch). An upstream probe on 9 held-out pre-2024 bugs found current docs cover 6/9 contracts (67%). The memorization canary (0/9) serves as the recall claim's positive control; a full rediscovery study against bug-present versions is future work.
- **Excluded set.** 17 of the 29 excluded submissions are Milvus; closed-no-label may reflect maintainer non-engagement rather than invalidity, so excluding them could hide an FP tail (bounded in §5.3: worst case 36/81=44.4%).
- **Single-layer counterfactual.** The 45.6% figure combines the maintainer-adjudicated 36/52 baseline with 27 live-re-probed FPs; triage might reclassify a few of the 27, and the arm is bounded to one feedback cycle.

6 Related Work

VDBMS testing. VDBFuzz [23] is the first dedicated VDBMS fuzzer (crash-oracle-based) and is complementary to this work. The roadmap [22] and the empirical bug study [25] define the agenda and taxonomy we build upon. MeTMaP [21] applies metamorphic testing to false vector matching in LLM-augmented generation. BUZZBEE [26] fuzzes general DBMSs.

REST API and schema testing. RESTler [3] infers producer–consumer dependencies among REST request types for stateful fuzzing; EvoMaster [2] evolves REST API test sequences. Schemathesis [8] applies property-based testing directly to OpenAPI/GraphQL schemas to surface contract violations, but requires a standards-compliant specification—which VDBMS REST endpoints do not serve (we probe `/swagger`, `/openapi.json`, and related paths in §5.3, all returning 404), so schema-driven fuzzers cannot be applied off-the-shelf. More recent REST fuzzers—foREST [13], MINER [14], and DynER [5]—advance sequence/parameter-value generation but remain OpenAPI-driven with 5XX/crash oracles; none targets semantic compliance against the documented contract. These tools target schema-conformance and crash at the API boundary; we extend the boundary to *semantic* compliance (does the response honor the documented contract?) and to VDBMS-specific invariants.

Database correctness oracles. NoREC [17] tests SQL via a non-optimizing reference engine; TLP [18] partitions queries three ways; DQE [19] pivots query execution; DDLCheck [20] targets DDL correctness. These assume reference SQL semantics absent in VDBMS APIs; our contract oracle replaces reference SQL with documentation-derived invariants.

LLM-based testing and verification. LLMs increasingly drive test generation and software-engineering tasks [10], and multi-agent LLM systems are surveyed comprehensively by He et al. [9]; we use multi-agent debate [7] for generation and judgment. A key difference from prior LLM-as-judge patterns is that we identify a self-confirmation failure mode when one model family both generates the contract and judges compliance—the contract-hallucination phenomenon (§4). The contract-hallucination failure mode we identify is a spec-extraction instance of LLM hallucination more broadly [11], mitigated in general by self-consistency [24]—we instantiate the mitigation as source-grounded falsification rather than prompt-level consensus. Concurrent LLM-driven REST testing (LlamaRestTest [12]) fine-tunes a small LLM for input generation and dependency detection; our delta is orthogonal—CTS targets contract hallucination via maintainer-authority falsification, not input quality.

Test oracles and contracts. The oracle problem is a long-standing concern [4]; our contract oracle is an instance of the *specified* class and the cosine-bounded / completeness subclass is an instance of the *derivable* class. Property-based testing [6] and Design by Contract [16] are precursors of specification-driven checking; static API-misuse detection [1] targets client-side misuse, whereas we target server-side non-compliance. General fuzzing is surveyed in [15].

7 Conclusion and Future Work

We presented TestVDB, the first LLM-driven system for detecting API compliance defects in VDBMSs, built on Contract-Truth Separation. On a controlled retrospective over 52 candidates, the dev-reviewer's source anchor lifts false-positive suppression from 31% to 81% while retaining 96.7% of true positives; aggregated maintainer-adjudicated precision is 69.2% (within [43.9%, 80.5%]). *Scope:* boundary/validation compliance, complementary to crash-focused fuzzing; result-correctness oracles (ANN recall) remain open. Contract hallucination propagation generalizes beyond VDBMSs: any system where one LLM family both extracts a spec and checks compliance is susceptible—REST API contract testing, configuration validation, and policy-as-code compliance wherever documentation is the only available oracle. CTS-style source-grounded falsification is the natural mitigation in each. Future work: stronger dev-reviewer backbones, an empirical VDBFuzz comparison, cross-library ablation beyond Milvus and Qdrant, and an end-to-end discovery-recall study.

References

[1] Sven Amann, Hoan Anh Nguyen, Sarah Nadi, Tien N. Nguyen, and Mira Mezini. 2020. A Systematic Evaluation of Static API-Misuse Detectors. *IEEE Transactions on Software Engineering* 46, 12 (2020), 1170–1188. doi:10.1109/TSE.2018.2827384

[2] Andrea Arcuri. 2021. Automated Black- and White-Box Testing of RESTful APIs with EvoMaster. *IEEE Software* 38, 3 (2021), 72–78. doi:10.1109/MS.2020.3013820

[3] Vaggelis Atlidakis, Patrice Godefroid, and Marina Polishchuk. 2019. RESTler: Stateful REST API Fuzzing. In *Proceedings of the IEEE/ACM International Conference on Software Engineering (ICSE)*. 748–758. doi:10.1109/ICSE.2019.00083

[4] Earl T. Barr, Mark Harman, Yue Jia, Alexandru Marginean, and Justyna Petke. 2015. The Oracle Problem in Software Testing: A Survey. *IEEE Transactions on Software Engineering* 41, 7 (2015).

[5] Juxing Chen, Yuanchao Chen, Zulie Pan, et al. 2024. DynER: Optimized Test Case Generation for RESTful API Fuzzers Guided by Dynamic Error Responses. *Electronics (MDPI)* 13, 17 (2024), 3476. doi:10.3390/electronics13173476

[6] Koen Claessen and John Hughes. 2000. QuickCheck: A Lightweight Tool for Random Testing of Haskell Programs. In *Proceedings of the ACM SIGPLAN International Conference on Functional Programming (ICFP)*.

[7] Yilun Du, Shuang Li, Antonio Torralba, Joshua B. Tenenbaum, and Igor Mordatch. 2023. Improving Factuality and Reasoning in Language Models through Multiagent Debate. arXiv preprint arXiv:2305.14325.

[8] Dmitry Dygalo. 2024. Schemathesis: Property-Based API Testing for OpenAPI and GraphQL. https://schemathesis.io. Accessed: 2026-07.

[9] Junda He, Christoph Treude, and David Lo. 2025. LLM-Based Multi-Agent Systems for Software Engineering: Literature Review, Vision, and the Road Ahead. *ACM Transactions on Software Engineering and Methodology (TOSEM)* 34, 5 (2025), 1–30. doi:10.1145/3712003

[10] Xinyi Hou, Yanjie Zhao, Yue Liu, et al. 2024. Large Language Models for Software Engineering: A Systematic Literature Review. *ACM Transactions on Software Engineering and Methodology (TOSEM)* 33, 5 (2024), 220:1–220:79. doi:10.1145/3695988

[11] Ziwei Ji, Nayeon Lee, Rita Frieske, et al. 2023. Survey of Hallucination in Natural Language Generation. *Comput. Surveys* 55, 12 (2023), 248:1–248:38. doi:10.1145/3571730

[12] Myeongsoo Kim, Saurabh Sinha, and Alessandro Orso. 2025. LlamaRestTest: Effective REST API Testing with Small Language Models. In *ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE)*.

[13] Jiaxian Lin, Tianyu Li, Yang Chen, et al. 2023. foREST: A Tree-based Black-box Fuzzing Approach for RESTful APIs. In *IEEE Int. Symp. on Software Reliability Engineering (ISSRE)*. 695–705. doi:10.1109/ISSRE59848.2023.00023

[14] Chenyang Lyu, Jie Xu, Shouling Ji, et al. 2023. MINER: A Hybrid Data-Driven Approach for REST API Fuzzing. In *USENIX Security Symposium*. 4517–4534.

[15] Valentin J. M. Manès, HyungSeok Han, Choongwoo Han, et al. 2021. The Art, Science, and Engineering of Fuzzing: A Survey. *IEEE Transactions on Software Engineering* 47, 11 (2021), 2312–2331. doi:10.1109/TSE.2019.2946563

[16] Bertrand Meyer. 1992. Applying "Design by Contract". *IEEE Computer* 25, 10 (1992).

[17] Manuel Rigger and Zhendong Su. 2020. Finding Bugs in Database Systems via Non-Optimizing Reference Engine Construction. In *Proceedings of the ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE)*.

[18] Manuel Rigger and Zhendong Su. 2020. Finding Bugs in Database Systems via Query Partitioning. *Proc. ACM Program. Lang.* 4, OOPSLA (2020), 211:1–211:30. doi:10.1145/3428279

[19] Manuel Rigger and Zhendong Su. 2020. Testing Database Engines via Pivoted Query Execution. In *Proceedings of the IEEE/ACM International Conference on Software Engineering (ICSE)*.

[20] Jiansen Song, Wensheng Dou, Yingying Zheng, Yu Gao, Ziyu Cui, Wei Wang, and Jun Wei. 2025. Detecting Schema-Related Logic Bugs in Relational DBMSs via Equivalent Database Construction. *Proceedings of the VLDB Endowment* 18 (2025), 2281–2293.

[21] Guanyu Wang, Yuekang Li, Yi Liu, Gelei Deng, Tianlin Li, Guosheng Xu, Yang Liu, Haoyu Wang, and Kailong Wang. 2024. MeTMaP: Metamorphic Testing for

Detecting False Vector Matching Problems in LLM Augmented Generation. In *Proceedings of the IEEE/ACM International Conference on AI Foundation Models and Software Engineering (FORGE)*.

[22] Shenao Wang, Yanjie Zhao, Yinglin Xie, Zhao Liu, Xinyi Hou, Quanchen Zou, and Haoyu Wang. 2025. Towards Reliable Vector Database Management Systems: A Software Testing Roadmap for 2030. arXiv preprint arXiv:2502.20812.

[23] Shenao Wang, Yanjie Zhao, Yinglin Xie, Zhao Liu, Xinyi Hou, Quanchen Zou, and Haoyu Wang. 2026. VDBFuzz: Understanding and Detecting Crash Bugs in Vector Database Management Systems. In *Proceedings of the IEEE/ACM International Conference on Software Engineering (ICSE)*.

[24] Xuezhi Wang, Jason Wei, Dale Schuurmans, et al. 2023. Self-Consistency Improves Chain of Thought Reasoning in Language Models. In *Proceedings of the International Conference on Learning Representations (ICLR)*.

[25] Yinglin Xie, Xinyi Hou, Yanjie Zhao, Shenao Wang, Kai Chen, and Haoyu Wang. 2025. Toward Understanding Bugs in Vector Database Management Systems. arXiv preprint arXiv:2506.02617.

[26] Yupeng Yang, Yongheng Chen, Rui Zhong, Jizhou Chen, and Wenke Lee. 2024. BUZZBEE: Towards Generic Database Management System Fuzzing. In *Proceedings of the USENIX Security Symposium (USENIX Security)*.